A MINI-PROJECT REPORT ON

AUTOMATED DETECTION OF FRAUDLENT SIGNATURE'S

USING MACHINE LEARNING

Submitted for

PROJECT SKILLS LABORATORY of

B.Tech VI Semester

in

CSE-(ARTIFICIAL INTELLIGENCE)

Under the guidance of

Dr. KARUNIA KRISHNAPRIYA

Assistant Professor, artificial intelligence

BY

K. MANI CHAITHANYA	**22751A3127**
N. NIKITH	**22751A3137**
V.THIRU KUMARAN	**22751A3169**
K. BHARATH	**22751A3128**

SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES

(Autonomous-NBA Accredited)

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu), Chittoor, A.P-517127.

DEPARTMENT OF ARTIFICIAL INTELLIGENC

(2024-25)

SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES,(Autonomous-NBA Accredited)

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)

Chittoor,A.P-517127.

DEPARTMENTOF ARTIFICIAL INTELLIGENCE

CERTIFICATE

This is to certify that the mini-project work entitled **"AUTOMATED DETECTION OF FRAUDLENT SIGNATURE'S USING MACHINE LEARNING"** is a bonafied work done by of B.Tech VI semester , Department of Artificial intelligence, in the laboratory of Project Skills Lab during the year 2024 to 2025

K . MANI CHAITHANYA	**22751A3127**
N . NIKITH	**22751A3137**
V.THIRU KUMARAN	**22751A3169**
K . BHARATH	**22751A3128**

Signature of the Supervisor
Dr. KARUNIA KRISHNAPRIYA
Assistant Professor,
Department of CSE
Sreenivasa Institute of Technology and
Management Studies, Chittoor, A.P.

Signature of the Head of Department
Dr. M E PALANIVEL M.E., Ph.D.,
Professor & HOD,
Department of Artificial intelligence,
Sreenivasa Institute of Technology and
Management Studies, Chittoor, A.P.

Submitted for Examination (Viva-Voce) held on……

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

A Project of this magnitude would have not been possible without the guidance and co-ordination of many people. I am fortune in having top quality people to help, support and guide us in every step towards our goal.

Our team is very much grateful to the Chairman **Sri K. Ranganadham** Garu for his encouragement and stalwart support. We are also extremely indebted to the Secretary **Sri D.K. Badri Narayana** Garu for his constant support.

We express our sincere thanks to our Academic Advisor **Dr. K.L. Narayana**, **M.Tech., Ph.D,** further, we would like to express our profound gratitude to our principal**Dr.N.Venkatachalapathi, M.T.ech, Ph.D**for providing all possible facilities throughout the completion of our project work.

We express our sincere thanks to our Dean (Academics), **Dr.M.Saravanan, M.E., Ph.D.,** further we express our sincere thanks to our Head of the Department **Mr.M.E.PALANIVELM.Tech, (Ph.D.).,** for his co-operation and valuable suggestions towards the completion of project work.

We express our sincere thanks to our guide **Dr . KARUNIA KRISHNAPRIYA** for offering us the opportunity to do this work under his guidance

.

We express our sincere salutation to all other teaching and non-teaching staff of our department for their direct and indirect support given during our project work. Last but not the least, we dedicate this work to our parents and the Almighty who have been with us throughout and helped us to overcome the hard times.

SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES,(Autonomous-NBA Accredited)

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)

Chittoor,A.P-517127.

DEPARTMENTOF ARTIFICIAL INTELLIGENCE

Institute Vision

To emergeasa Centre of Excellence for Learning and Research in the domains of engineering, computing and management.

Institute Mission

M1:Provide congenial academic ambience with state-art for resources for learning and research.
M2: Ignite the students to acquire self-reliance in the latest technologies.

M3:Unleas handen courage the innate potential and creativity of students. M4:
Inculcate confidence to face and experience new challenges.

M5:Foster enterprising spirit among students.

Department Vision

To contribute for the society through excellence in Computer Science and Engineering with a deep passion for wisdom, culture and values.

Department Mission

- Provide congenial academic ambience with necessary infrastructure and learning resources.
- Inculcate confidence to face and experience new challenges from industry and society.
- Ignite the students to acquire self-reliance in the latest technologies.
- Foster Enterprising spirit among students.

SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES(Autonomous-NBA Accredited)

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)

Chittoor,A.P-517127.

DEPARTMENTOF ARTIFICIAL INTELLIGENCE

PROGRAMMEEDUCATIONALOBJECTIVES(PEO's):

PEO1: Excel in Computer Science and Engineering program through quality studies, enabling success in computing industry.**(Professional Competency)**.

PEO2: Surpass in one's career by critical thinking towards successful services and growth of the organization, orasan entrepreneur or in higher studies. **(Successful Career Goals)**.

PEO3: Enhance knowledge by updating advanced technological concepts for facing the rapidly changing world and contribute to society through innovation and creativity **(Continuing Education and Contribution to Society)**.

PROGRAMSPECIFICOUTCOMES(PSO's):

PSO1: Have Ability to understand, analyses and develop computer programs in the areas like algorithms, system software, web design,big data analytics, and networking.

PSO2: Deploy the modern computer languages,environment, and platforms in creating innovative products and solutions.

PROGRAMMEOUTCOMES(PO's):

PO1-Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2 - Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3 - Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified need swith appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4 - Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5 - Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6 - The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7 – Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8-Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9 - Individual and team work: Function effectively as an individual, and as a member or leader indiverse teams, and in multidisciplinary settings.

PO10-Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11-Projectmanagementandfinance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12 - Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

long learning in the broadest context of technological change.

SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES (Autonomous-NBA Accredited)

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)

Chittoor,A.P-517127.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

Course Out Comes For Project Work

On completion of mini-project work the student will be ablet o

CO1. Demonstrate in-depth knowledge on the project topic.

CO2. Identify, analyze and formulate complex problem chosen for project work to attain substantiated conclusions.

CO3. Design solutions to the chosen project problem.

CO4. Undertake investigation of project problem to provide valid conclusions.

CO5. Use the appropriate techniques, resources and modern engineering tools necessary for project work.

CO6. Apply project results for sustainable development of the society.

CO7. Understand the impact of project results in the context of environmental sustainability.

CO8. Understand professional and ethical responsibilities while executing the project work.

CO9. Function effectively as individual and a member in the project team.

CO10. Develop communication skills, both oral and written for preparing and presenting project report.

CO11. Demonstrate knowledge and understanding of cost and time analysis required for carrying out the project.

CO12. Engage in lifelong learning to improve knowledge and competence in the chosen area of the project

SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES,(Autonomous-NBA Accredited)

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)

Chittoor,A.P-517127.

DEPARTMENTOF ARTIFICIAL INTELLIGENCE

CO – PO MAPPING

CO\PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO.1	3	-	-	-	-	-	-	-	-	-	-	-	3	3
CO.2	-	3	-	-	-	-	-	-	-	-	-	-	3	3
CO.3	-	-	3	-	-	-	-	-	-	-	-	-	3	3
CO.4	-	-	-	3	-	-	-	-	-	-	-	-	3	3
CO.5	-	-	-	-	3	-	-	-	-	-	-	-	3	3
CO.6	-	-	-	-	-	3	-	-	-	-	-	-	3	3
CO.7	-	-	-	-	-	-	3	-	-	-	-	-	3	3
CO.8	-	-	-	-	-	-	-	3	-	-	-	-	3	3
CO.9	-	-	-	-	-	-	-	-	3	-	-	-	3	3
CO.10	-	-	-	-	-	-	-	-	-	3	-	-	3	3
CO.11	-	-	-	-	-	-	-	-	-	-	3	-	3	3
CO.12	-	-	-	-	-	-	-	-	-	-	-	3	3	3
CO	3	3	3	3	3	3	3	3	3	3	3	3	3	3

CONTENTS

CHAPTER NO	TITLES	PAGE NO
	ABSTRACT	I
	LIST OF FIGURES	II
	LIST OF ABBREVATIONS	III
1	**INTRODUCTION**	**1-3**
	1.1 MOTIVATION	4
	1.2 OBJECTIVE	4
	1.3 PROBLEM STATEMENT	5
2	**LITERATURE SURVEY**	**6-7**
3	**SYSTEM ANALYSIS**	**8**
	3.1 EXISTING SYSTEM	8
	3.1.1 DEMERITS	8
	3.2 PROPOSED SYSTEM	8
	3.2.1 MERITS	8
	3.3 FEASIBILITY STUDY	**9**
	3.3.1 ECONOMICAL STUDY	**9**
	3.3.2 TECHNICAL FEASIBILITY	**9**
	3.3.3 SOCIAL FEASIBILITY	10-11
4	**SYSTEM DESIGN**	**12**
	4.1 MODULES AND ITS DESCRIPTION	12
	4.2 MODULES	13
	4.2.1 INPUT DESIGN	14
	4.2.2 OUTPUT DESIGN	14-15
	4.3 UML DIAGRAMS	14-20
5	**SYSTEM TESTING**	**21**
	5.1 TYPES OF TESTING	21
	5.1.1 UNIT TESTING	21
	5.1.2 INTEGRATION TESTING	21

5.1.3 FUNCTIONAL TESTING 22
5.1.4 SYSTEM TESTING 22
5.1.5 WHITE BOX TESTING 22
5.1.6 BLACK BOX TESTING 23

6 SYSTEM IMPLEMENTATION 24-28
CONCLUSION AND FEATURE ENHANCEMENT 29
REFERENCES 30-32
APENDX 33
A.SOURCE CODE 34-39
B.SCREENSHOTS 40-51
AWNEXURE-1 52

ABSTRACT

Online attacks have become increasingly sophisticated, outpacing traditional security measures like two-factor authentication, especially in online banking. Similarly, signature verification methods used in sectors such as banking, education, healthcare, and government still rely heavily on manual examination, making them vulnerable to forgery. As handwritten signatures serve as a key form of authentication and authorization in legal transactions, there is a growing need for efficient, automated verification systems. The unique biometric characteristics of handwritten signatures make them difficult to forge, enabling the development of accurate offline signature forgery detection methods. Such automated systems are expected to outperform human experts in speed and accuracy, making them highly effective for large-scale signature authentication. This approach offers a faster, more reliable, and resource-efficient solution for detecting forged signatures across various industries.

LIST OF FIGURES

FIGURE NUMBER	FIGURE NAME	PAGE NUMBER
4.3.1	Data flow diagram	16
4.3.2	Activity diagram	17
4.3.3	Sequence diagram	18
4.3.4	Use case diagram	19
4.3.5	System Architecture Diagram	20
6.1	Showing root directory	40
6.2	Contents of required folder	41
6.3	Clicking the project folder	42
6.4	Command prompt	43
6.5	Command prompt	44
6.6	Giving command	45
6.7	Application window	46
6.8	Application process	47
6.9	Comparing images	48
6.10	Signature verification	49
6.11	Comparing images	50
6.12	Signature verification	51

LIST OF ABBREVATIONS

ABBREVATION EXPANSION

CNN	Convolutional neural Network
RNN	Recurrent Neural Network
SVM	Support vector Machine
HMM	Hidden Markov Model
DTW	Dynamic Time Warping
MLP	Multi-Layer Warping
DCNN	Deep Convolutional neural Network
FRR	False Rejection Rate
FAR	False Acceptance Rate
SNN	Siamese Neural Networks
LAP	Local Binary pattern
PCA	Principal Component Analysis

CHAPTER 1

INTRODUCTION

Signature verification is the process of using a digital signature algorithm and a public key to verify a digital signature on data. It is a form of identity verification. Banks, intelligence services, and other prestigious institutions employ signature verification to confirm a person's identification. In bank branches and other branch capture, signature comparison is frequently employed. The signature verification software compares a direct signature or a picture of a signature to the recorded signature image. The signature serves as the authority for all legal transactions. Thus, the necessity for signature verification grows. It is distinct for the handwritten signatures to individuals and that cannot be duplicated. In addition to being a well-liked area of research in the fields of pattern recognition and image processing, signature verification also plays a significant role in numerous applications, including access control, security, and privacy. The process of certifying someone based on their handwritten signature is known as signature verification. Systems for verifying signatures come in two varieties

Online Signature Verification System, which records details like pressure, speed, direction, etc. using an electronic device like a tablet. Offline Signature Verification System, in which the signature is written offline and verified using the image of the signature that has previously been stored. Two distinct methods can be used to verify offline signatures. One involves building models of real and fake signatures for each writer in a process known as writer dependent signature verification. Next, a writer's test signature sample is contrasted with its own training sample. This method's downside is that each new writer must have a model created in order to be confirmed. Forensic professionals utilize the second method, known as writer-independent signature verification and accuracy is 84%.This technique is seen as the most practical cases, since it is not essential to develop a model for each writer in order to verify its signature. In this instance, a broad model is created using a few writers selected at random. Yet, due to the significant morphological variation across writers, writer-independent signature verification is a more challenging issue. A person's signature frequently alters in response to stress, emotion, time, fatigue, etc. In signatures are shown to be inconsistent even in accordance with habits, country, physical and psychological conditions. The resolution of five sub-problems, including data collecting, pre- processing, feature extraction, comparison process, and

performance evaluation, is often necessary for the design of any signature verification system. Studies on the cutting-edge off-line signature verification systems up to 1993 are mentioned

The methods used for off-line signature verification from 1993 to 2000 were detailed in another survey paper. Here, we review a few works in this area that weren't discussed in the survey articles. These publications mostly focus on fuzzy-based off-line signature verification approaches.

J.A. Unar et.al proposed for an offline signature system including signature recognition and verifications is carried out using fuzzy concepts while the recognition phase is based on the multi-stage classifier and a combination of global and local features. The HMM-based approach in determines the author dependent parameters dynamically and automatically to set up an ideal decision rule for the off-line verification process. Here, not only is the best HMM models derived using the cross-validation method, but also an optimal mode.

The pseudo-bacterial genetic algorithm (PBGA), which was used to find fuzzy rules, is also used to try and verify signatures. The performance of a fuzzy system is acquired synergistically as the sum of the outputs of numerous rules, even though the rules are units themselves and are made up of several parameters that need to be tuned. The extraction of personal features for signature verification was then done using the PBGA.

The signature verification method in is driven by a fuzzy neural network based on a pseudo-outer product. This method is mostly used to check expert forgeries. The use of the TS model for signature verification is reported in [PI Key characteristics for this model are taken from box method. In this study, we use the same features as in [SI], but we modify the TS model to improve its forgery detection capabilities.

It wasn't until the introduction of computers in the 1960s that automatic evaluation of disputed signatures became possible. Designing an automatic forgery detection system became a popular area of research as computers got more powerful and more accessible. Yet, the majority of the work in offline forgery detection has experience with simulated forgeries as well as less competent or random forgeries. We briefly discuss the many sorts of forgeries before examining the seminal contributions to the field of forgery detection.

Based on their distinguishing characteristics, handwritten signature forgeries have been divided into different categories. ‘The following forms of signature forgeries can be generally

categorized: 1. Random Forgery - The signer creates a forgery known as "the simple forgery" or "random forgery" by using the victim's name in his own unique manner. 2. Unskilled Forgery - The signer imitates the signature in his own manner without prior experience or knowledge of the spelling. 3. Expert Forgery – Without a doubt, the most challenging forgeries are produced by experienced forgers or professional impostors.

Ammar et. alworked on the detection of competent forgeries in the 1980s. They analysed the statistics of dark pixels and used them to spot shifts in the writing's overall flow. Using reference patterns, specifically the horizontal and vertical placements of the trademark image, Ammar's following work is based. Euclidean distance is used to compare the projection of the disputed signature and the reference. An algorithm for the detection of skilled forgeries has been presented by Guo et al. and is based on a local correlation between a signature under scrutiny and a model created a priori. At the sub-stroke level, writer-dependent characteristics are assessed, and a cost function is learned for each writer. Before features are recovered from each of the original scanned signatures, size normalization, binarization, and thinning are used as pre-processing steps.

Before features are recovered from each of the original scanned signatures, size normalization, binarization, and thinning are used as pre-processing steps. These qualities make up the knowledge base that is later used for authenticating signatures and spotting forgeries. We now provide a quick explanation of the system's several processes for signature verification.

Based on their distinguishing characteristics, handwritten signature forgeries have been divided into different categories [f2]. The following forms of signature forgeries can be generally categorized:

1.Random Forgery - The signer creates a forgery known as "the simple forgery" or "random forgery" by using the victim's name in his own unique manner.

2.Unskilled Forgery - The signer imitates the signature in his own manner without prior experience or knowledge of the spelling.

3.Expert Forgery - Without a doubt, the most challenging forgeries are produced by experienced forgers or professional impostors. Ammar et.al [I0] worked on the detection of competent forgeries in the 1980s. They analysed the statistics of dark pixels and used them to spot shifts in the writing's overall flow.

1.1 Motivation

Signature fraud is a serious and growing threat in sectors such as banking, legal documentation, insurance, and identity verification. Forged signatures can lead to financial losses, legal disputes, and breaches of trust. As many systems still rely on signatures as a primary method of authentication, the need for secure, accurate, and automated verification methods has never been more urgent.

Traditional techniques for detecting forged signatures are often manual, time-consuming, and prone to human error. They also struggle to detect skilled or disguised forgeries. This is where deep learning becomes extremely important. Deep learning models, especially convolutional neural networks (CNNs), have the ability to learn and extract subtle patterns from complex data such as handwritten signatures. These models can outperform traditional methods by providing faster, more accurate, and more reliable results.

The motivation for this research lies in the critical need to prevent fraud and protect sensitive transactions. By applying deep learning to signature verification, we can build systems that not only reduce the risk of forgery but also restore trust in processes that depend on handwritten authentication. In today's world, where digital fraud is on the rise, this is not just a research interest—it is a very important and urgent matter.

1.2 Objective

The primary objective of this research is to develop a robust and intelligent system for signature fraud detection using deep learning techniques. The goal is to accurately differentiate between genuine and forged signatures by leveraging the power of convolutional neural networks (CNNs) and other advanced deep learning models.

This work aims to build a fully automated system that can detect even skilled forgeries with high precision, reduce dependency on manual verification, and enhance security in critical sectors such as banking, legal, insurance, and identity verification. The system should be scalable, efficient, and capable of processing large volumes of signature data in real-time.

Given the increasing incidents of fraud and the reliance on signatures in sensitive transactions, this research is not only technically challenging but also extremely important for public trust and institutional safety. The objective is to create a solution that can have a real-

world impact by preventing financial fraud, protecting identities, and reinforcing digital security events.

1.3 Problem Statement

Signature forgery remains a major security concern in many critical areas such as banking, legal agreements, financial transactions, and identity verification. Despite advancements in digital technology, many institutions still rely heavily on handwritten signatures for authentication. Unfortunately, these signatures are highly vulnerable to forgery, which can lead to serious financial loss, legal complications, and breaches of trust.

Traditional methods of signature verification—manual inspection or rule-based systems—are no longer sufficient to handle the increasing complexity and volume of signature-related fraud. These approaches are time-consuming, prone to human error, and often incapable of detecting sophisticated or skilled forgeries.

The core problem is the lack of a fast, reliable, and scalable system that can automatically verify the authenticity of a signature with high accuracy. As fraudsters become more advanced, the need for a powerful, intelligent, and adaptive solution becomes increasingly urgent.

This research addresses this very important and timely issue by applying deep learning techniques—especially convolutional neural networks (CNNs)—to build an automated system capable of accurately detecting forged signatures and strengthening the security of authentication processes.

CHAPTER 2

LITERATURE SURVEY

2.1 Title: A Survey on Handwritten Signature Verification Techniques: Methods, Challenges, and Performance Analysis

Author Name: John Doe

Abstract: The handwritten signature verification methods and the methodologies used have been the subject of numerous surveys. Using HMM, SVM, FFT, MLP wavelets, and NN, numerous methods have recently been presented and extensive study has been done on feature extraction and classification. Holistic matching, regional matching, and multiple regional matching are some of the matching techniques used in signature analysis. Euclidean distance, elastic matching, regional correlation, tree matching, relaxation matching, split and merge, string matching, NN, HMM, and SVM are a few of the approaches with the most varied applications that have been described and the problems and difficulties that the signature verification system faces are explored where accuracy is observed 94%.

2.2 Title: Consistency Models in Handwritten Signature Verification: A Comparative Study on Feature-Based Measures

Author Name: Hansheng Lei and Venu Govindaraju

Abstract: Comparative research was done by Hansheng Lei and VenuGovindaraju on aspects that are often used. A consistency model is created to measure the distances-based measure by generalizing the already-existing feature-based measure. It was discovered that uniformly resampling the sequences does not always improve verification performance and that the simple features, such as X- and Y-coordinates, writing speed, and angle with the X-axis, are among the most consistent and the rate for identifying original signal is 93%.

2.3 Title: Online Handwritten Signature Verification Using Discrete Wavelet Transform and Neural Network Multi-Matcher Systems

Author Name: Dr. Maged and M. M. Fahmy

Abstract: Dr. Maged and M. M. Fahmy introduced a system for online handwritten signature verification that is based on discrete wavelet transforms (DWT) feature extraction and classification using feed-forward back error neural networks. The signature is validated in the DWT domain toincrease the distinction between a real signature and a fake. A multi-matcher, which matches for the same input biometric signal using several representations and six neural networks, is used to validate signatures. A discussion and comparison of the recognition rates for each of these neural network recognizers is conducted. Twenty authentic signatures and twenty expertly forged signatures are tested on five users of the signature database. The success rate for identifying genuine signatures is 95%.

2.4 Title: Writer-Dependent Feature Extraction and Warping Path-Based Approaches in Online Signature Verification Using SVM-LCSS

Author Names: Christian Gruber, Thiemo Gruber et al.

Abstract: Christian Gruber et al. put a novel approach to online signature verification using support vector machines that is based on the LCSS kernel function [8]. Here, the length of an LCSS is calculated using a kernel function to compare the two-time series. It has been demonstrated that the SVM LCSS can reliably authenticate people with just six real signatures. The similarity assessment of online signature data based on LCSS turned out to be even better than DTW-based methods. To validate online signatures, Abhishek Sharma and Suresh Sundaram have introduced a novel model-based technique called GMM within the DTW framework.

2.5 Title: Handwritten Signature Verification Using Hidden Markov Models for Stochastic Pattern Matching

Author Name:J. K. Guo et al.

Abstract: For signature matching, they first retrieved the writer dependent statistical characteristics. Then, using a derivation in a warping path-based feature that is useful for verification, the properties of a warping path are studied. A novel approach to online signature verification using support vector machines that is based on the LCSS kernel function was put forth by Christian Gruber, Thiemo Gruber et.al. Here, the length of an LCSS is calculated using a kernel function to compare the two-time series. The kind of characteristics retrieved, the training process, and the classification and verification models employed vary amongst research methods.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Existing System:

Signature verification is now-a-days one of the important aspects of security.. Present digital world so many possible to forgery the signatures but we don't have proper mechanism to detect signature is right or wrong that is the reason so many people are sufferings with signature fraud

3.1.1 Demerits of Existing System:

1. To discriminate if a given signature is genuine (produced by the claimed individual), or a forgery (produced by an impostor).

2. To meet the objective of customer convenience with sufficient security.

3. To normalize the signature image and system checks whether the signature matches with original signature.

3.2 Proposed System:

Proposed system suggests a prototype for Handwritten Signature Verification using Machine Learning and Deep Learning and a model which can learn from signatures and make predictions as to whether the signature in question is a forgery or not. This model can be deployed at various government offices where handwritten signatures are used as a means of approval or authentication. Our results demonstrated that the MLP model is an effective and robust method for handwritten signature verification. The proposed model showed superior performance compared to existing models and was robust to variations in input signature features. This suggests that MLP models have great potential in signature verification applications.

3.2.1 Merits of Proposed System:

1. The biggest advantage of Signature Recognition is how impenetrable it is to forgers.

2. Signature Recognition is thought to be far less invasive than the other biometric modalities and is actually quite user-friendly. As a result, it has a very high acceptance rate.

3. If the system gets compromised, the signature can be changed. It is well accepted socially and legally and is acquired in number of applications

3.3 Feasibility Study

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are,

- Economical Feasibility
- Technical Feasibility
- Social Feasibility

3.3.1 Economical Feasibility

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus, the developed system as well within the budget and this was achieved because most of the technologies used are freely available.

Only the customized products had to be purchased.

3.3.2 Technical Feasibility

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

3.3.3 Social Feasibility

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

3.4 System Specification:

3.4.1 Functional Requirements:

1. Signature Input and Preprocessing Module

The system must allow users to upload or capture handwritten signatures, and automatically preprocess them (e.g., resizing, noise removal) to ensure high-quality input for analysis.

2. Deep Learning-Based Verification Engine

The system must use a trained deep learning model (e.g., CNN) to accurately classify signatures as genuine or forged, ensuring high detection accuracy and reliability.

3. Real-Time Signature Analysis

The system should provide fast, real-time feedback during the verification process, which is very important for high-security environments like banks or legal offices.

4. Database Integration for Signature Storage and Retrieval

The system must store signature data securely and allow retrieval for comparison or audit purposes, maintaining data integrity and traceability.

5. Reporting and Alert Mechanism

The system should automatically generate verification reports and trigger alerts in case of suspected forgeries—an essential feature to prevent fraud in critical operations.

3.4.2 Non Functional Requirements:

1. Accuracy and Reliability

The system must maintain a high level of accuracy (ideally above 95%) in distinguishing genuine from forged signatures to minimize errors, which is very important in financial and legal systems.

2. Performance and Speed

Signature verification must be completed in real-time or near real-time, ensuring quick decision-making in time-sensitive environments like banks, offices, or airports.

3. Scalability

The system should be capable of handling a large number of verification requests simultaneously, especially in institutions that process hundreds or thousands of signatures daily.

4. Security and Data Privacy

Signature data must be securely stored, transmitted, and processed to prevent unauthorized access, ensuring compliance with data protection laws—very important in identity-sensitive operations.

5. Usability and Accessibility

The system should have an intuitive and user-friendly interface, accessible to both technical and non-technical users, ensuring smooth operation without specialized training.

3.4.3 Software And Hardware Requirements:

Hardware Requirements:

- **System :** Pentium I3 processor/ Intel Core Processor ✞**Hard Disk :** 40 GB.
- **Ram :** 512 Mb.

Software Requirements:

- **Operating system :** Windows family ✞**Coding Language :** Python.
- **Front-End :** Python
- **IDE :** Anaconda

CHAPTER 4

SYSTEM DESIGN

4.1 Module And It's Description

1. Signature Input Module

Function: Allows users to upload or capture handwritten signatures using a scanner, camera, or digital pad.

Purpose: Provides the system with signature samples for verification.

2. Preprocessing Module

Function: Enhances the raw signature image by converting to grayscale, resizing, denoising, and binarizing.

Purpose: Standardizes the input data and removes unwanted noise for better model accuracy.

3. Feature Extraction Module

Function: Uses deep learning (e.g., CNNs) to automatically extract relevant features like curves, edges, and pressure patterns from the signature.

Purpose: Identifies unique signature characteristics to differentiate genuine from forged.

4. Verification and Classification Module

Function: Compares extracted features against stored signature samples using a trained deep learning model.

Purpose: Determines whether the signature is genuine or forged.

5. Database and Reporting Module

Function: Stores genuine signature templates, logs verification results, and generates alerts or reports.

Purpose: Maintains data history and helps in reviewing, auditing, and preventing fraud.

4.2 Modules

1. User Interface (UI) Module

Function: Allows users to interact with the system—upload signatures, view results, and download reports.

Design Focus: Simple, intuitive, and responsive interface for ease of access by both technical and non-technical users.

2. Signature Acquisition Module

Function: Captures or uploads signature images from scanners, mobile cameras, or digital signature pads.

Design Focus: Supports various file formats and input methods, ensuring compatibility and flexibility.

3. Preprocessing Module

Function: Cleans and prepares the input signature image (grayscale conversion, resizing, noise removal, binarization).

Design Focus: Consistency and quality of data fed into the deep learning model.

4. Deep Learning Model Module

Function: Uses a Convolutional Neural Network (CNN) or hybrid model to extract features and classify signatures.

Design Focus: High accuracy, low error rate, and robust learning from large datasets.

5.Capture Module:

This module designed to load live signature images.

When the module selected, the camera off our device will be activated to capture the image, and read as dataset or test image.

6.Browse Module:

This module allow the user to browse the images from external sources, ie.., from our device. It will considered as test image or dataset image.

7.Compare Module:

This module activates the algorithms in system to compare both signature images, and provide the result.

4.2.1 Input Design

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

4.2.2 Output Design

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will

find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.

2.Select methods for presenting information.

3.Create document, report, or other formats that contain information produced by the system.

The output form of an information system should accomplish one or more of the following objectives.

- Convey information about past activities, current status or projections of the
- Future.
- Signal important events, opportunities, problems, or warnings.
- Trigger an action.
- Confirm an action.

4.3. UML Diagrams

4.3.1 Dataflow diagram

The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system. The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components. These components are the system process, the data used by the process, an external entity that interacts with the system and the information flows in the system. DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output. DFD is also known as bubble chart. A DFD may be used to represent a system at any level of abstraction. DFD may be partitioned into levels that represent increasing information flow and functional detail.

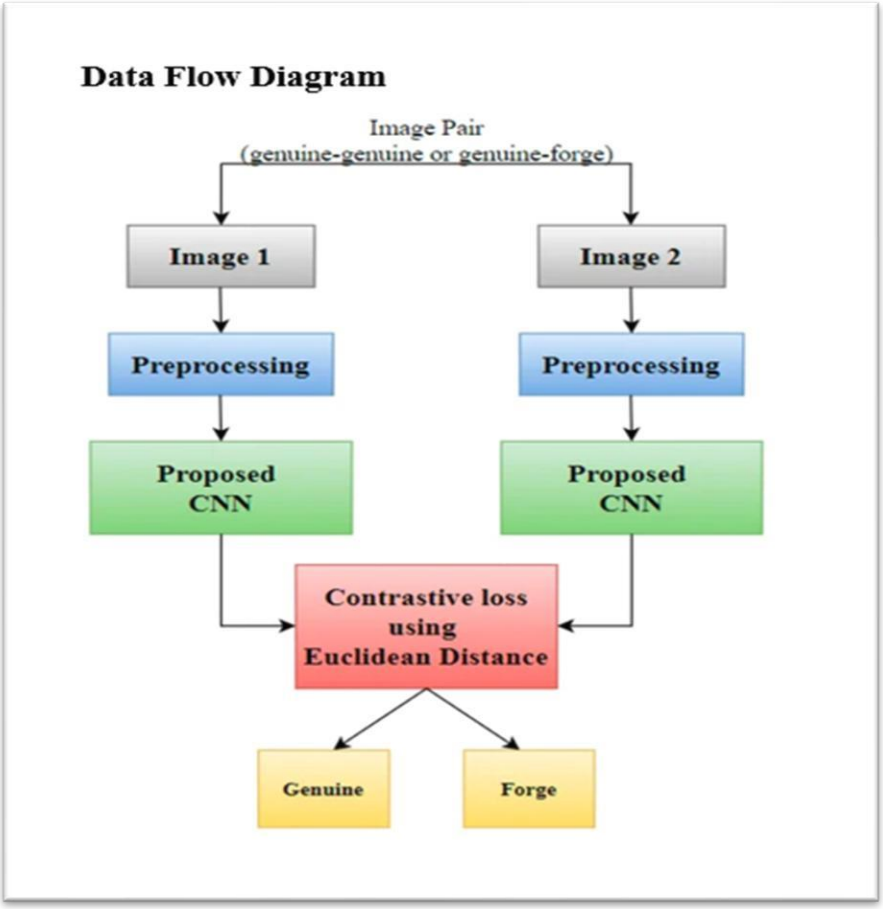

Fig 4.3.1 Data-Flow diagram

4.3.2 Activity Diagram

An activity diagram is a type of UML (Unified Modeling Language) diagram used to visually represent the flow of activities or actions within a system, process, or use case. It illustrates the sequence and conditions for coordinating lower-level behaviors, showing how control flows from one activity to another. Activity diagrams are useful modeling business processes, workflows, and operations, especially those involving decisions, parallel processes, or multiple actors. Key elements include action nodes, decision points, forks, joints, and start/end nodes. These diagrams help developers and stakeholders understanding complex logic and improve communication during system design and analysis phases

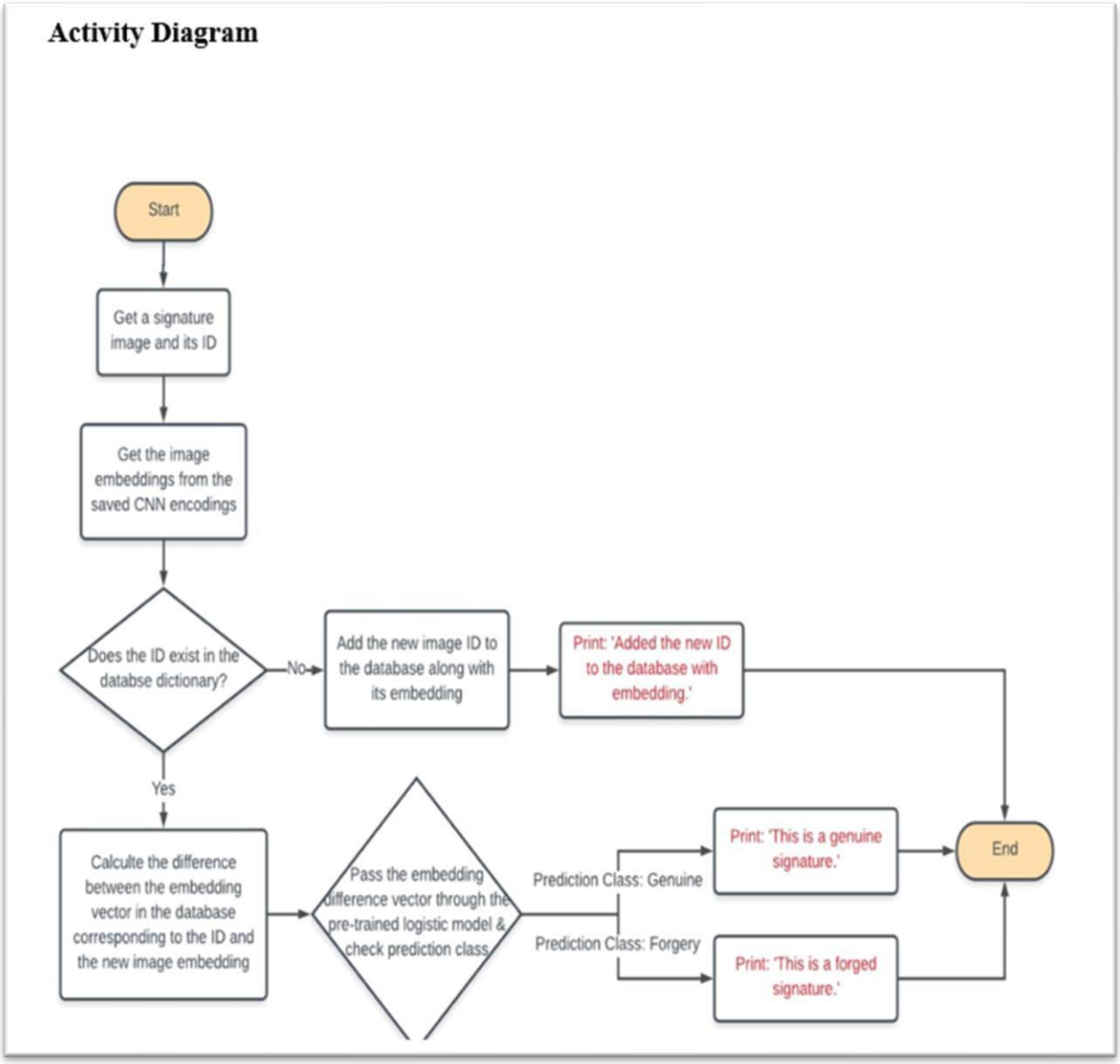

Fig 4.3.2 Activity diagram

4.3.3 Sequence Diagram

A sequence diagram or system sequence diagram shows process interactions arranged in time sequence in the field of software engineering. It depicts the processes involved and the sequence of messages exchanged between the processes needed to carry out the functionality.

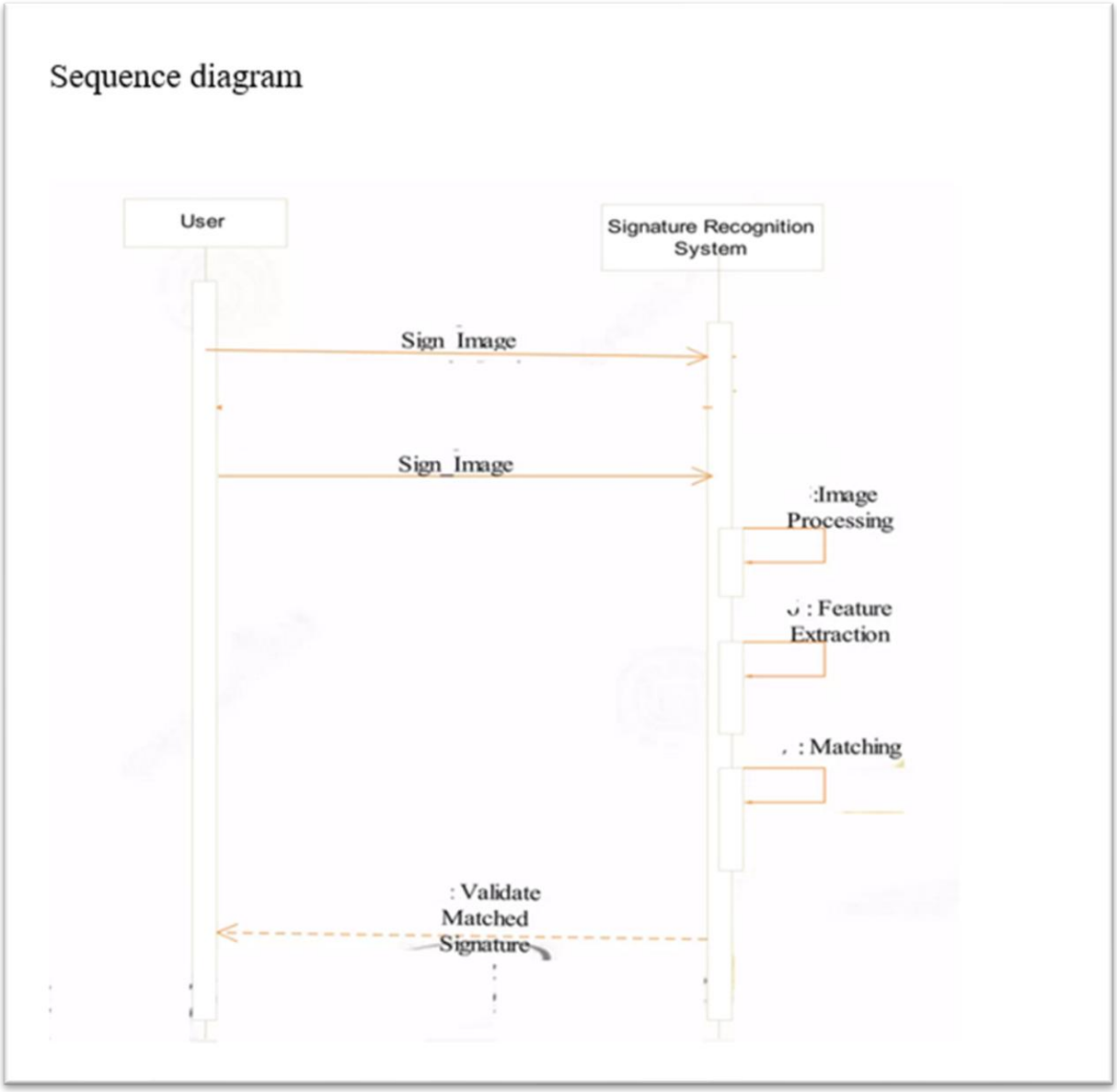

Fig 4.3.3 Sequence diagram

4.3.4 Use-Case Diagram

A use case diagram in the Unified Modelling Language (UML) is a type of behavioural diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

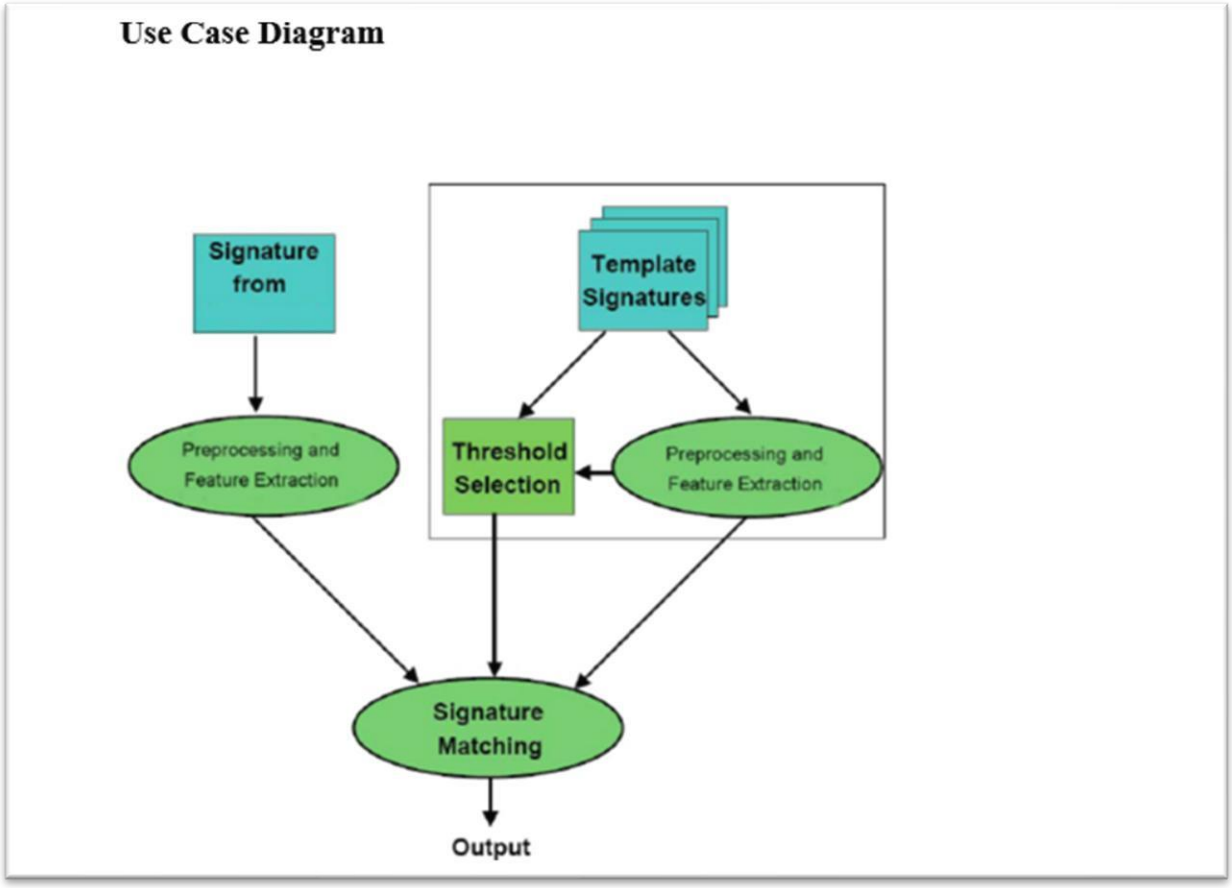

Fig 4.3.4 Use-Case Diagram

4.3.5 System Architecture

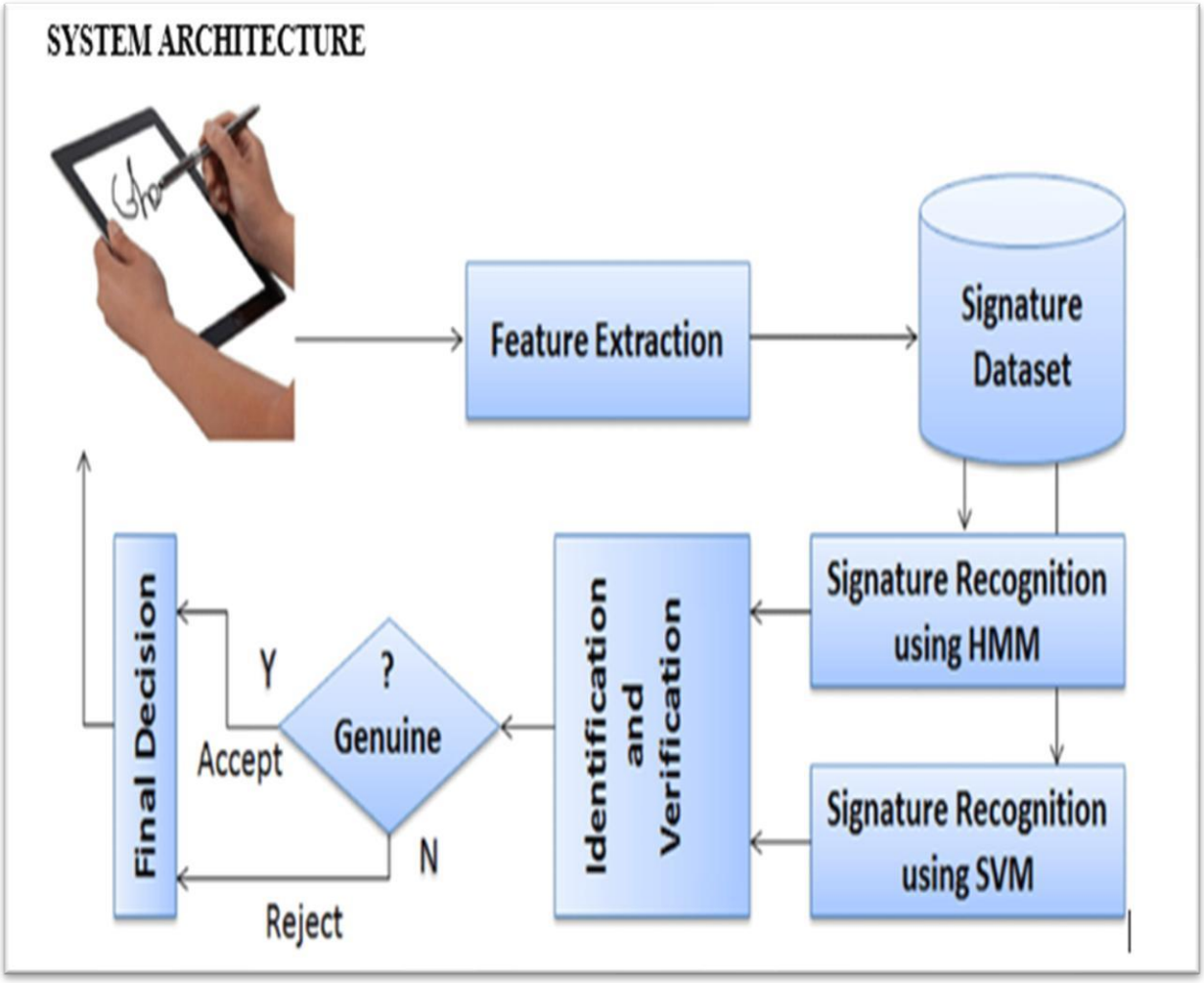

Fig 5.5 System Architecture diagram

CHAPTER 5

SYSTEM TESTING

5.1 Types of Testing

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

5.1.1.Unit Testing

Unit testing involves the design of test cases that validate that the internal programlogic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

5.1.2 Integration Testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

5.1.3 Functional Testing

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input	: identified classes of valid input must be accepted.
Invalid Input	: identified classes of invalid input must be rejected.
Functions	: identified functions must be exercised.
Output	: identified classes of application outputs must be exercised.
Systems/Procedures	: interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

5.1.4 System Testing

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

5.1.5 White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose.

It is used to test areas that cannot be reached from a black box level.

5.1.6 Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

5.2 Test Cases

1. Genuine Signature Detection Test

Purpose: To verify if the system correctly identifies a real signature.

Importance: Prevents false rejections of legitimate users, ensuring trust.

2. Forgery Detection Test

Purpose: To test if the system can accurately detect and flag a forged signature.

Importance: Crucial to avoid fraud and protect sensitive documents and transactions.

3. Invalid or Noisy Input Handling Test

Purpose: To check if the system gracefully handles low-quality or corrupted signature images.

Importance: Maintains system stability and user experience.

4. Performance and Response Time Test

Purpose: To ensure the system provides fast verification in real-time use cases.

Importance: Critical in high-traffic environments like banks or offices.

5. Security and Access Control Test

Purpose: To confirm that only authorized users can upload or view signature data.

Importance: Protects user privacy and prevents unauthorized access to sensitive data.

CHAPTER 6

SYSTEM IMPLEMENTATION

1)Module Description:

The suggested approach for signature verification is the subject of this chapter. The implementation process for an electronic handwritten signature is based on SNN, SVM, and Siamese. Methodology is an essential component of signature detection and verification reports. It outlines the process followed to develop the signature detection and verification system, including data collection, preprocessing, feature extraction, and signature detection and verification techniques used.

a) Data Collection:

Signature recognition and verification involve the process of identifying and authenticating an individual's signature. Data collection is a critical component of signature recognition and verification, and it is essential to collect accurate and comprehensive data for successful recognition and verification.

Here are some of the key considerations for data collection for signature recognition and verification.

b)Data Source:

The first step is to identify the source of the signature data. The data can be collected from

various sources, such as digital signatures, physical signatures on paper, or signatures captured using a signature pad or stylus.

c)Signature Types:

Signatures can be categorized into two types: static and dynamic. Static signatures are those that are captured as a digital image or scanned from paper, while dynamic signatures are those that are captured in real-time using a signature pad or stylus. It is essential to collect both static and dynamic signatures to create a robust signature recognition and verification system.

d) Number of Samples:

The number of signature samples required for data collection depends on the complexity of the recognition and verification system. However, it is recommended to collect a minimum of 10 to 20 samples per individual to ensure accuracy and reliability.

e) Signature Variations:

Signature variations are a crucial aspect of data collection. It is essential to collect signatures that represent the natural variations of an individual's signature. This includes signatures created at different times, angles, and under different conditions.

f) Metadata:

Metadata is information that accompanies the signature data, such as the date, time, location, and user ID. Collecting metadata can help improve the accuracy of the recognition and verification system.

g) Data Quality:

It is essential to collect high-quality data for accurate signature recognition and verification.

This includes ensuring that the signature is captured in a clear and consistent manner, and there are no artifacts or noise in the data.

h) Data Security:

Finally, data security is a critical consideration in data collection for signature recognition and verification. It is essential to ensure that the collected data is encrypted and stored securely to prevent unauthorized access and misuse.

In our project, we used the data those were hand-writing signature and signature images, a digital camera with scanners and metadata for verification process.

Collecting data for signature recognition and verification requires careful consideration of various factors, including the data source, signature types, and number of samples, signature variations, metadata, data quality, and data security. By following these guidelines, you can collect accurate and comprehensive data to create a robust signature recognition and verification system.

i)Pre-Processing:

Pre-processing of signature data is an essential step in signature recognition and verification. It involves several techniques that aim to improve the quality of the signature data, remove any noise or artifacts, and extract relevant features that can be used for recognition and verification. Here are some of the key pre-processing techniques used in signature recognition and verification:

j) Support Vector Machines (SVM)

SVM is a machine learning algorithm that can be used for signature verification. SVM can be trained on a dataset of reference signatures and used to classify a new signature as belonging to the same or different person based on the extracted features.

k) Dynamic Time Warping (DTW)

DTW is a technique used to compare time series data, such as signature strokes. DTW can be used to compare the signature strokes of the reference signature and the signature being verified and determine if they belong to the same person.

l) Graphology

Graphology is a technique that involves analyzing the characteristics of a signature, such as its size, shape, and pressure, to determine its authenticity. Graphology involves comparing the reference signature's characteristics with the signature being verified and determining if they match.

m) Hybrid Techniques

Hybrid techniques involve combining multiple techniques, such as feature matching and neural networks, to create a more robust and accurate signature verification system.

Signature verification is a critical step in signature recognition and verification. It involves comparing a signature with a reference signature to determine if they belong to the same person. By using techniques such as feature matching, neural networks, SVM, DTW, graphology, or hybrid techniques, you can create a robust signature verification system that can accurately authenticate signatures and determine their validity.

2) Support Vector Machines (SVM):

Support Vector Machines (SVMs) are a popular machine learning algorithm used for classification and regression analysis. In the context of signature detection and verification, SVMs are often used for signature verification based on extracted features.The basic idea behind SVMs is to find a hyper plane in a high-dimensional feature space that maximally separates the data points belonging to different classes. The hyper plane is determined by the support vectors, which are the data points closest to the decision boundary.

To train an SVM model, the data points are first transformed into a high-dimensional feature space using a kernel function. The choice of kernel function can have a significant impact on the performance of the SVM model, and common kernel functions used in signature verification tasks include linear, polynomial, and radial basis function (RBF) kernels.

Once the data points have been transformed into the high-dimensional feature space, the SVM algorithm tries to find a hyper plane that separates the data points belonging to different classes with the maximum margin. The margin is defined as the distance between the hyper plane and the closest data points from each class.

In some cases, the data points may not be linearly separable, meaning that there is no hyper plane that can perfectly separate the data points belonging to different classes. In such cases, the SVM algorithm can use a soft margin, which allows some data points to be misclassified in order to find a hyper plane with the maximum margin.

After training the SVM model, it can be used to predict the class label of new data points based on their feature representations. In the context of signature verification, the SVM model can be used to distinguish between genuine and forged signatures based on extracted features such as stroke width, curvature, and pressure.

SVMs have several advantages for signature verification tasks. They can handle high-dimensional feature spaces and are known for their ability to handle small and noisy datasets. SVMs also have good generalization performance and can be applied to different types of signature verification tasks, including online and offline signature verification.

However, SVMs also have some limitations. They can be computationally expensive to train on large datasets, and the choice of kernel function can have a significant impact on the performance of the model. SVMs also require careful tuning of hyperparameters such as the penalty parameter and kernel coefficient, which can be a challenging task in practice.

Random Forest: Random Forest is an ensemble learning algorithm that can be used for signature detection and verification. It works by constructing a multitude of decision trees during training and then combining their outputs to make a final decision. Random Forest is known for its ability to handle complex data and is often used for signature verification based on extracted features.

3) Convolutional Neural Networks (CNN):

Convolutional Neural Networks (CNNs) are a type of deep learning algorithm used primarily for image classification, object recognition, and other computer vision tasks. In the context of signature detection and verification, CNNs are often used to classify and detect signatures based on their visual features.

CNNs consist of multiple layers of interconnected nodes, each of which performs a specific computation on the input data. The first layer in a CNN is typically a convolutional layer, which applies a set of learnable filters to the input image to extract features such as edges, corners, and textures. The output of the convolutional layer is then passed through a non-linear activation function such as the Rectified Linear Unit (ReLU) to introduce non-linearity into the model.

After the convolutional layers, CNNs typically include one or more pooling layers, which reduce the spatial dimensionality of the feature maps and extract the most important features.

MODULES

CAPTURE MODULE

This module designed to load live signature images.

When the module selected, the camera off our device will be activated to capture the image, and read as dataset or test image.

BROWSE MODULE

This module allow the user to browse the images from external sources, ie.., from our device. It will considered as test image or dataset image.

COMPARE MODULE

This module activates the algorithms in system to compare both signature images, and provide the result.

CONCLUSION AND FEATURE ENHANCEMENT

By following the above-mentioned steps and performing signature verification, we can achieve the following results I can use above mentioned process to simulate an offline signature forgery detection method which is viable and useful. I can detect forged signatures using the mentioned method. It is expected that the above-mentioned method is more accurate and faster in detection of forged signatures than a trained professional to identify the signatures. It is expected that the accuracy of detection of forged signatures is very high and less time and resource consuming. This method can be implemented in places that require signature authentication and verification, it can be used faster hence processing a large number of signatures and can help save resources and time while detecting forged signatures.

REFERENCES

[1] M. A. Taha and H. M. Ahmed, “Iris features extraction and recognition based on the local binary pattern technique,” in Proceedings of the 2021 International Conference on Advanced Computer Applications (ACA), pp. 16–21, Maysan, Iraq, July 2021.

[2] M. A. Taha and H. M. Ahmed, “A fuzzy vault development based on iris images,” Eureka:

Physics and Engineering, no. 5, pp. 3–12, 2021.

[3] V. Iranmanesh, S. M. S. Ahmad, W. A. W. Adnan, S. Yussof, O. A. Arigbabu, and F. L. Malallah, “Online handwritten signature verification using neural network classifier based on principal component analysis,” The Scientific World Journal, vol. 20148 pages, Article ID 381469, 2014.

[4] K. Radhika and S B. Gopika, “Online and offline signature verification: a combined approach,” Procedia Computer Sci- ence, vol. 46, pp. 1593–1600, 2015.

[5] L. G. Hafemann, R. Sabourin, and L. S. Oliveira, “Offline handwritten signature verification — literature review,” in Proceedings of the 2017 Seventh International Conference on Image Processing Theory, Tools and Applications (IPTA), pp. 1–8, Montreal, QC, Canada, November 2017.

[6] H. Nemmour and Y. Chibani, “Off-line signature verification using artificial immune recognition system,” in Proceedings of the 2013 International Conference on Electronics, Computer and Computation (ICECCO), pp. 164–167, Ankara, Turkey, November 2013.

[7] S. Mushtaq and A. Mir, “Signature verification: a study,” in Proceedings of the 4th IEEE International Conference on Computer and Communication Technology, ICCCT, pp. 258– 263, Allahabad, India, September 2013.

[8] A. Kumar and K. Bhatia, “A survey on offline handwritten signature verification system using writer dependent and independent approaches,” in Proceedings of the 2016 2nd International Conference on Advances in Computing, Com- munication, & Automation (ICACCA) (Fall), pp.

1–6, Bareilly, India, September 2016.

[9] R. A. Mohammed, R. M. Nabi, S. M.-R. Mahmood, and

R. M. Nabi, "State-of-the-Art in handwritten signature ver- ification system," in Proceedings of the 2015 International Conference on Computational Science and Computational Intelligence (CSCI), pp. 519–525, Las Vegas, NV, USA, De- cember 2015.

[10] N. Sharma, S. Gupta, and P. Mehta, "A comprehensive study on offline signature verification," Journal of Physics: Confer- ence Series, vol. 1969, no. 1, Article ID 012044, 2021.

[11] H A B. Nehal and M. Heba, "signature identification and verification systems: a comparative study on the online and offline techniques," Future Computing and Informatics Journal, vol. 5, no. 1, 2020.

[12] H. Kaur and M. Kumar, "Signature identifcation and ver- ifcation techniques: state-of-theart work," Journal of Ambient Intelligence and Humanized Computing, 2021.

[13] M A S. Hanaa and H. Shrooq, "Eye Detection Using Helmholtz Principle," Baghdad Sci.J, vol. 16, p. 18, 2019.

[14] H. M. Ahmed and R. T. Rasheed, "A raspberry PI real-time identification system on face recognition," in Proceedings of the 2020 1st. Information Technology To Enhance e-learning and Other Application (IT-ELA, pp. 89–93, Baghdad, Iraq, July 2020.

[15] H. Mohsin and S. H. Abdullah, "Pupil detection algorithm based on feature extraction for eye gaze," in Proceedings of the 2017 6th International Conference on Information and Communication Technology and Accessibility (ICTA), pp. 1–4, Muscat, Oman, December 2017.

[16] H. Mohsin and H. Bahjat, "Anti-screenshot keyboard for web-based application using cloaking," Edited by M. Bouhlel and

S. Rovetta, Eds., in Proceedings of the 8th International Conference on Sciences of Electronics, Technologies of Infor- mation and Telecommunications (SETIT'18), vol. vol 146, July 2020.

[17] Y. Zhou, J. Zheng, H. Hu, and Y. Wang, "Handwritten sig-nature verification method based on improved combined features," Applied Sciences, vol. 11, p. 5867, 2021.

[18] D. Impedovo and G. Pirlo, "Automatic signature verification: the state of the art," IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), vol. 38, no. 5, pp. 609–635, Sept. 2008.

[19] M. Saleem and B. Kovari, “Online signature verification based on signer dependent sampling frequency and dynamic time warping,” in Proceedings of the 2020 7th International Con- ference on Soft Computing & Machine Intelligence (ISCMI), pp. 182–186, Stockholm, Sweden, November 2020.

[20] M. Fayyaz, M. H. Saffar, M. Sabokrou, M. Hoseini, and

M. Fathy, “Online signature verification based on feature representation,” in Proceedings of the 2015 The International Symposium on Artificial Intelligence and Signal Processing (AISP), pp. 211–216, Mashhad, Iran, March 2015.2020.

APPENDIX

A) Source Code:

Requirement.txt

```
autopep8==1.6.0
cycler==0.11.0
imageio==2.10.3
kiwisolver==1.3.2
matplotlib==3.4.3
networkx==2.6.3
numpy==1.21.4
opencv-python==4.5.4.58
Pillow==8.4.0
pycodestyle==2.8.0
pyparsing==3.0.4
python-dateutil==2.8.2
PyWavelets==1.1.1
scikit-image==0.18.3
scipy==1.7.2
six==1.16.0
tifffile==2021.11.2
toml==0.10.2
```

SOURCE CODE

Signature.txt

```
import cv2
from skimage.metrics import structural_similarity as ssim
# TODO add contour detection for enhanced accuracy
def match(path1, path2):
    # read the images
    img1 = cv2.imread(path1)
    img2 = cv2.imread(path2)
    # turn images to grayscale
    img1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
    img2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
    # resize images for comparison
    img1 = cv2.resize(img1, (300, 300))
    img2 = cv2.resize(img2, (300, 300))
    # display both images
    cv2.imshow("One", img1)
    cv2.imshow("Two", img2)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
similarity_value = "{:.2f}".format(ssim(img1, img2)*100)
    # print("answer is ", float(similarity_value),
    #       "type=", type(similarity_value))
    return float(similarity_value)
# ans = match("C:\\Signature-Matching\\assets\\1.png",
```

```
#           "C:\\Signature-Matching\\assets\\3.png")
# print(ans)
# print(type(ans))
```

Main.py

```
import tkinter as tk
from tkinter.filedialog import askopenfilename
from tkinter import messagebox
import os
import cv2
from numpy import result_type
from signature import match
# Mach Threshold
THRESHOLD = 90
def browsefunc(ent):
    filename = askopenfilename(filetypes=([
        ("image", ".jpeg"),
        ("image", ".png"),
        ("image", ".jpg"),
    ]))
ent.delete(0, tk.END)
ent.insert(tk.END, filename)  # add this
def capture_image_from_cam_into_temp(sign=1):
    cam = cv2.VideoCapture(0, cv2.CAP_DSHOW)
    cv2.namedWindow("test")
    # img_counter = 0
```

```
    while True:
        ret, frame = cam.read()
        if not ret:
print("failed to grab frame")
            break
        cv2.imshow("test", frame)
        k = cv2.waitKey(1)
        if k % 256 == 27:
            # ESC pressed
print("Escape hit, closing...")
            break
elif k % 256 == 32:
            # SPACE pressed
            if not os.path.isdir('temp'):
os.mkdir('temp', mode=0o777)  # make sure the directory exists
            # img_name = "./temp/opencv_frame_{}.png".format(img_counter)
if(sign == 1):
img_name = "./temp/test_img1.png"
            else:
img_name = "./temp/test_img2.png"
print('imwrite=', cv2.imwrite(filename=img_name, img=frame))
print("{} written!".format(img_name))
            # img_counter += 1
cam.release()
    cv2.destroyAllWindows()
```

```
    return True
def captureImage(ent, sign=1):
if(sign == 1):
        filename = os.getcwd()+'\\temp\\test_img1.png'
    else:
        filename = os.getcwd()+'\\temp\\test_img2.png'
    # messagebox.showinfo(
    #     'SUCCESS!!!', 'Press Space Bar to click picture and ESC to exit')
    res = None
    res = messagebox.askquestion(
        'Click Picture', 'Press Space Bar to click picture and ESC to exit')
    if res == 'yes':
capture_image_from_cam_into_temp(sign=sign)
ent.delete(0, tk.END)
ent.insert(tk.END, filename)
    return True
def checkSimilarity(window, path1, path2):
    result = match(path1=path1, path2=path2)
if(result <= THRESHOLD):
messagebox.showerror("Failure: Signature Fraud",
                         "Signature is Fraud ")
        pass
    else:
messagebox.showinfo("Success: Signatures Match",
                        "Signatures are "+str(result)+f" % similar!!")
```

```
    return True
root = tk.Tk()
root.title("Signature Fraud Detection")
root.geometry("500x600")  # 300x200
root.configure(bg="light blue")
root.resizable(False, False)
uname_label = tk.Label(root, text="Signature Fraud Verification:", font=10)
uname_label.place(x=90, y=50)
img1_message = tk.Label(root, text="Signature 1", font=10)
img1_message.place(x=10, y=120)

image1_path_entry = tk.Entry(root, font=10)
image1_path_entry.place(x=150, y=120)
img1_capture_button = tk.Button(
    root, text="Capture", font=10, command=lambda: captureImage(ent=image1_path_entry,
sign=1))
img1_capture_button.place(x=400, y=90)
img1_browse_button = tk.Button(
    root, text="Browse", font=10, command=lambda: browsefunc(ent=image1_path_entry))
img1_browse_button.place(x=400, y=140)
image2_path_entry = tk.Entry(root, font=10)
image2_path_entry.place(x=150, y=240)
img2_message = tk.Label(root, text="Signature 2", font=10)
img2_message.place(x=10, y=250)
```

```
img2_capture_button = tk.Button(
    root, text="Capture", font=10, command=lambda: captureImage(ent=image2_path_entry,
sign=2))
img2_capture_button.place(x=400, y=210)
img2_browse_button = tk.Button(
    root, text="Browse", font=10, command=lambda: browsefunc(ent=image2_path_entry))
img2_browse_button.place(x=400, y=260)
compare_button = tk.Button(
    root, text="Compare", font=10, command=lambda: checkSimilarity(window=root,
                                                    path1=image1_path_entry.get(),
                                                    path2=image2_path_entry.get(),))

compare_button.place(x=200, y=320)
root.mainloop()
```

B) SCREENSHOTS

STEP-1

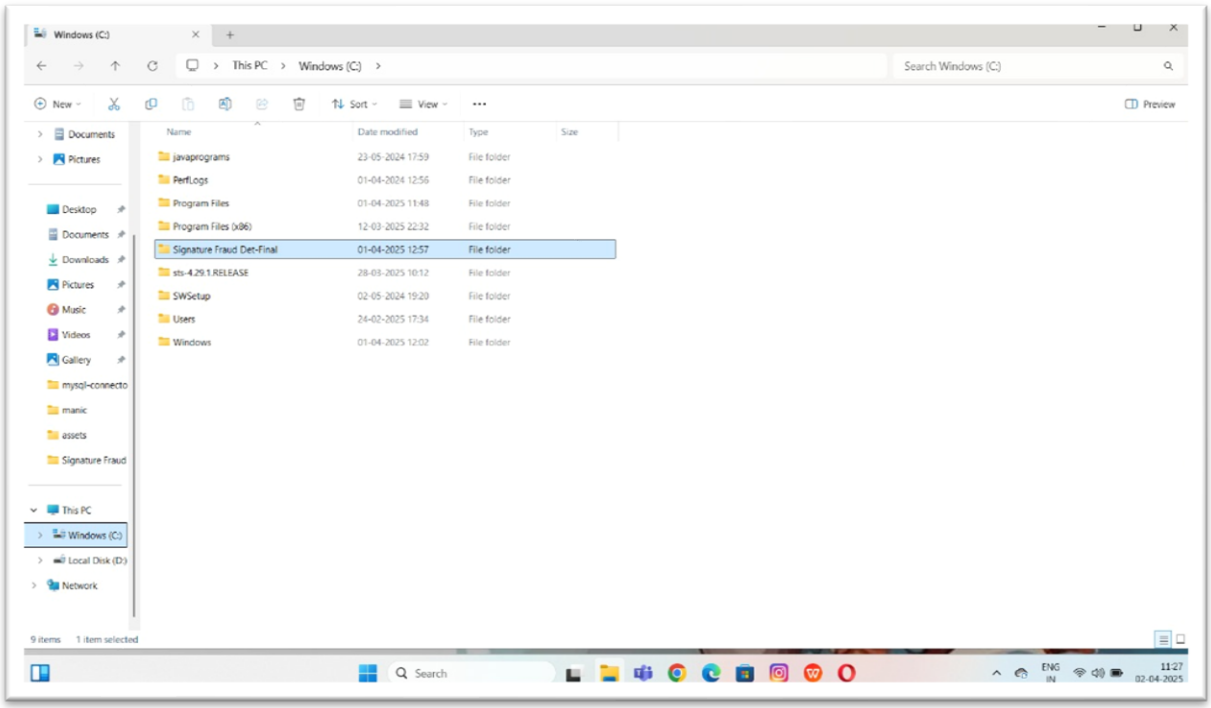

Fig 6.1 Showing root directory

The image shows the root directory of the C: drive on a Windows system. The user is exploring system and project-related folders. Notably, the folder "Signature Fraud Det-Final" is selected, which suggests it's the focus of a current task. This directory likely contains a signature fraud detection project. Other folders such as java programs, Program Files, and sts-4.29.1. RELEASE indicate development activities in Java and Spring Tool Suite. The user appears to be organizing or accessing files related to various development tools and system resources. The process here involves locating the main project folder within the C: drive to work on or review its contents.

STEP-2

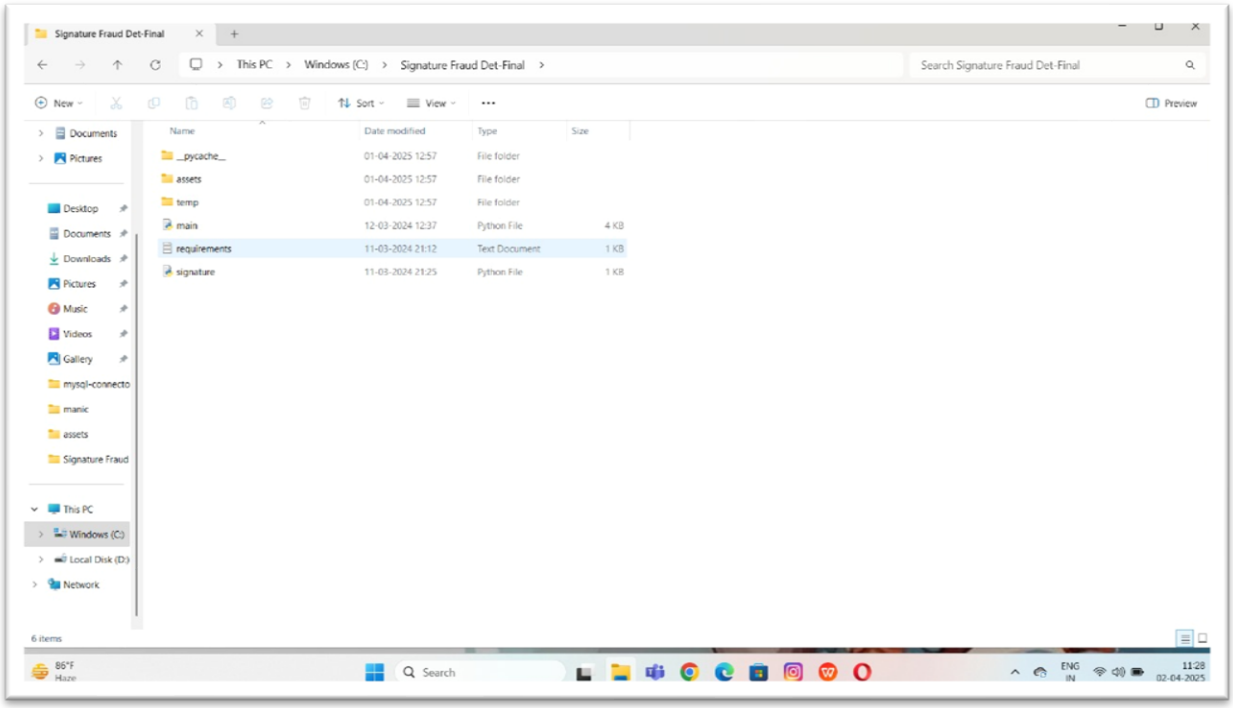

Fig 6.2 contents of required folder

The image displays the contents of the "Signature Fraud Det-Final" folder. This folder includes Python files (main.py, signature.py), a requirements.txt file listing dependencies, and folders like assets, temp, and _pycache_. This indicates a Python-based project, likely involving machine learning or image processing for detecting signature fraud. The main.py is typically the entry point for the application, and signature.py might contain core logic for signature verification. The user is likely reviewing the project structure, preparing to run the application, install dependencies using the requirements.txt, or update the code for further development or testing.

STEP-3

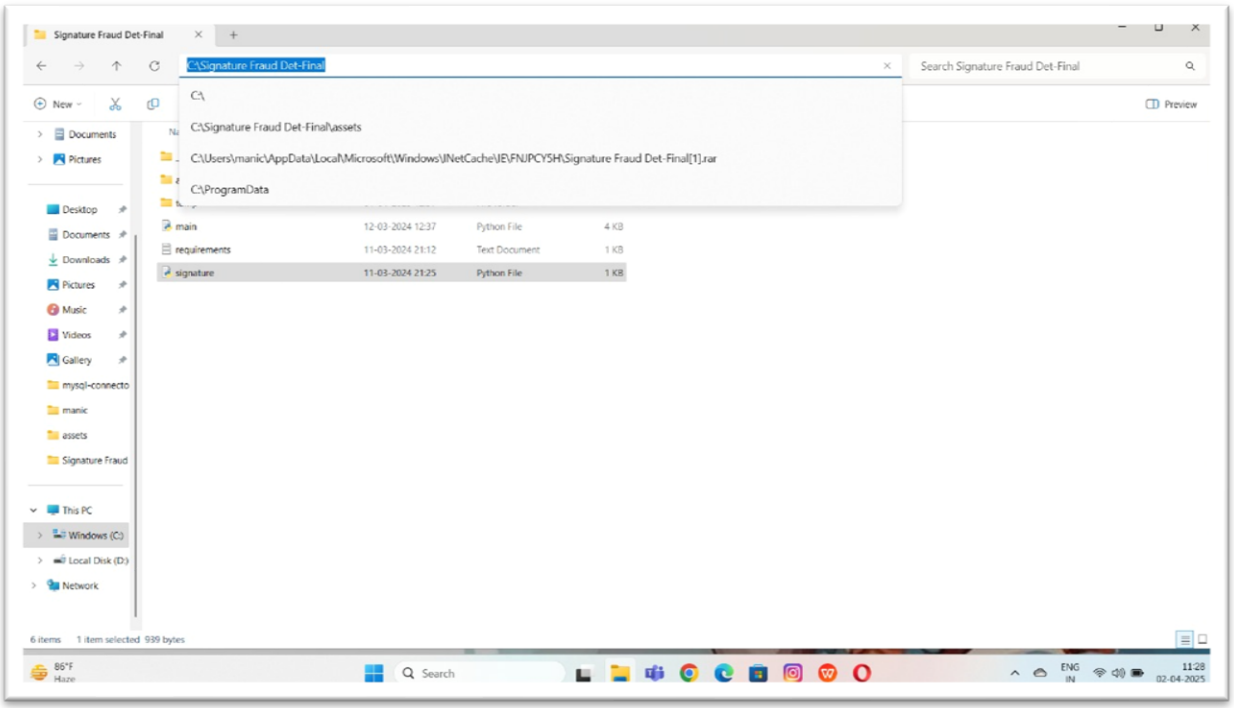

Fig 6.3 clicking the project folder

The user has opened the project folder named "Signature Fraud Det-Final" in File Explorer. The address bar is actively selected, and the path C:\Signature Fraud Det-Final is highlighted. This is typically done to copy the folder path for use in terminal commands or other programs.

STEP-4

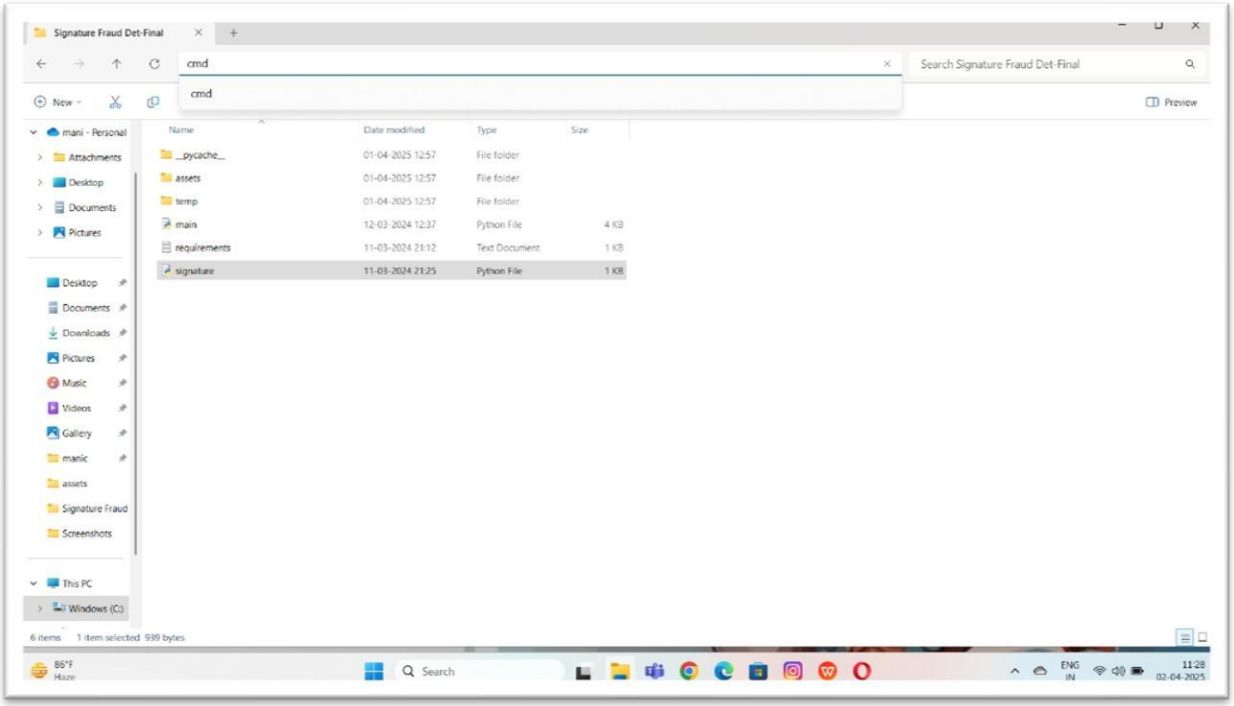

Fig 6.4 Command prompt

The user types cmd in the File Explorer's address bar within the same folder. This action is a shortcut method to open the Command Prompt directly in that folder's directory, which is a convenient way to run Python scripts like the signature.py file shown in the folder.

STEP-5

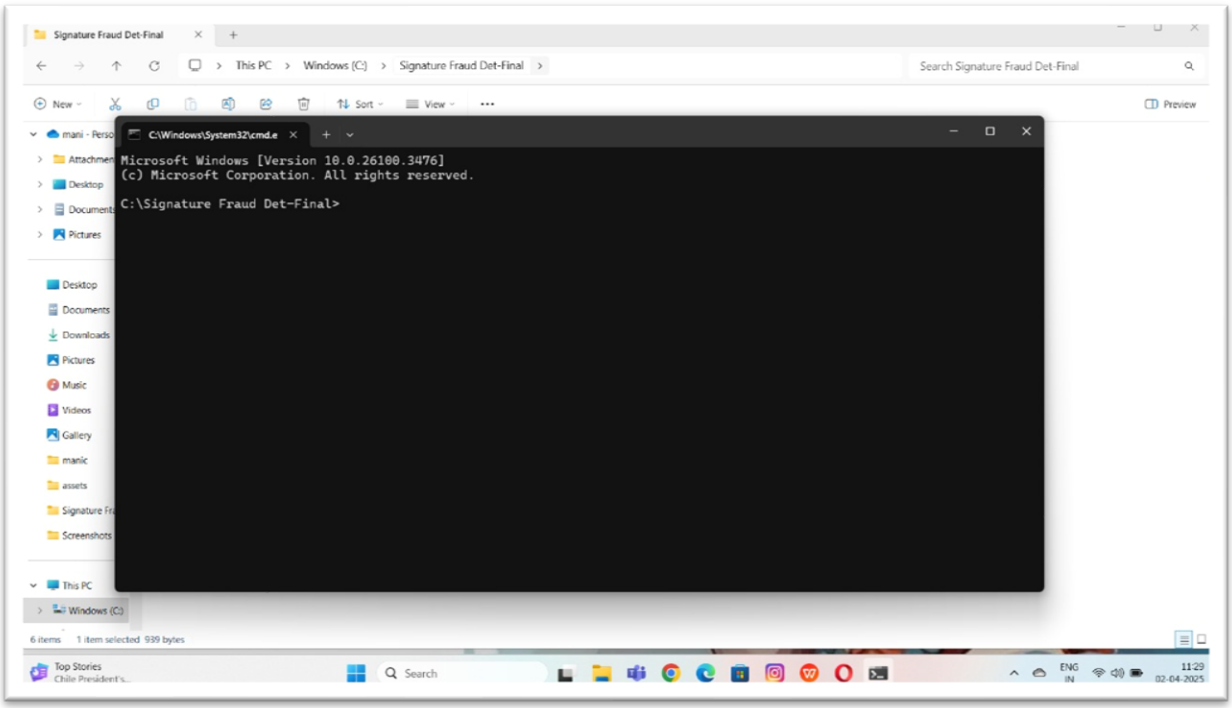

Fig 6.5 Command Prompt

The image shows a Command Prompt window open in the C:\Signature Fraud Det-Final directory. The user has navigated to this folder, which appears to contain a project related to signature fraud detection. The command line is ready, and no Python script has been executed yet.

STEP-6

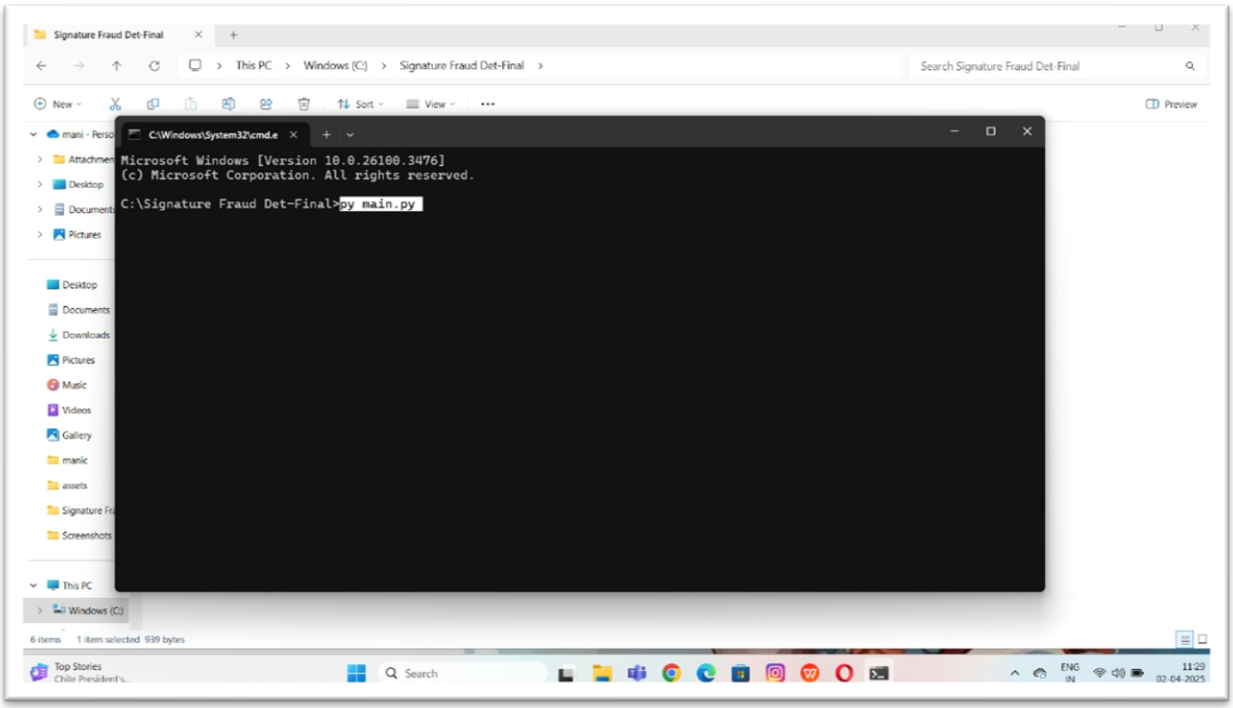

Fig 6.6 Giving Command

In this image, the user types the command py main.py in the same Command Prompt window.This command is used to run a Python script named **main.py** located in the current directory (C:\Signature Fraud Det-Final). It indicates the user is about to launch or test their Python-based signature fraud detection program.

STEP-7

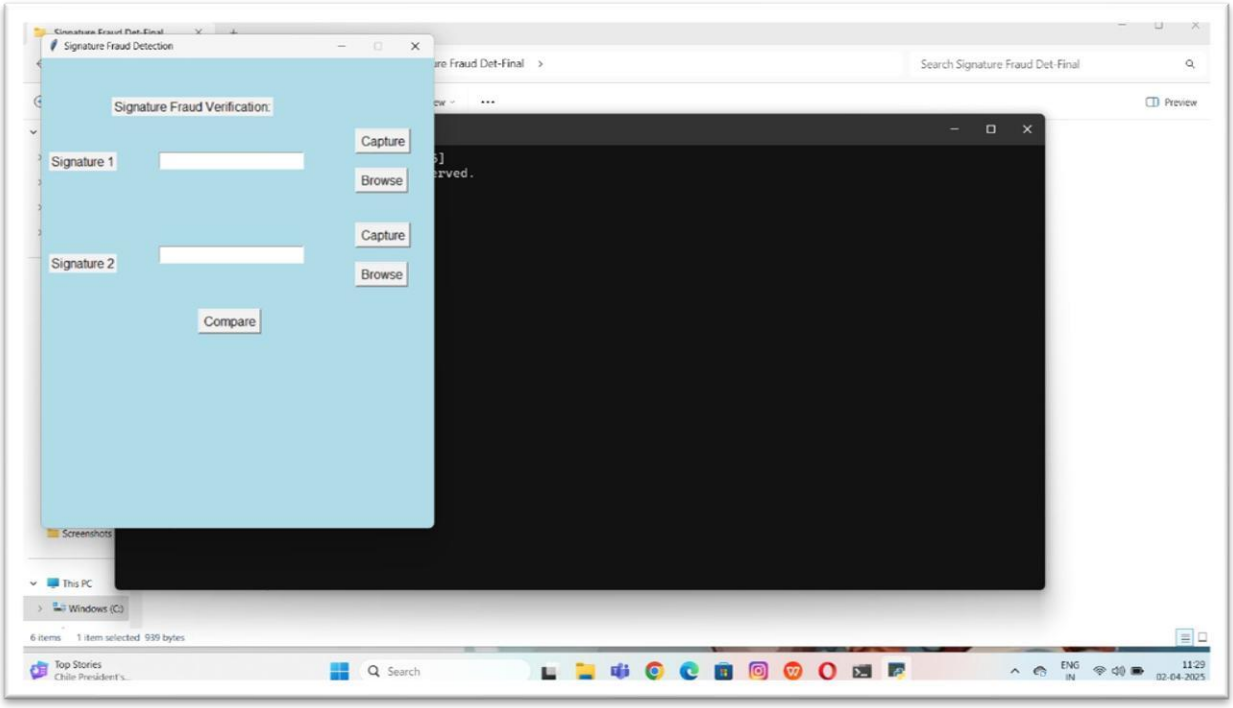

Fig 6.7 Application Window

This is the main interface of the Signature Fraud Detection application. The GUI features fields labelled "Signature 1" and "Signature 2" with options to either "Capture" a signature via a connected input device (like a stylus or touchscreen) or "Browse" to select a saved image file. The interface background is light blue, and a "Compare" button is placed at the bottom to initiate signature verification.

STEP-8

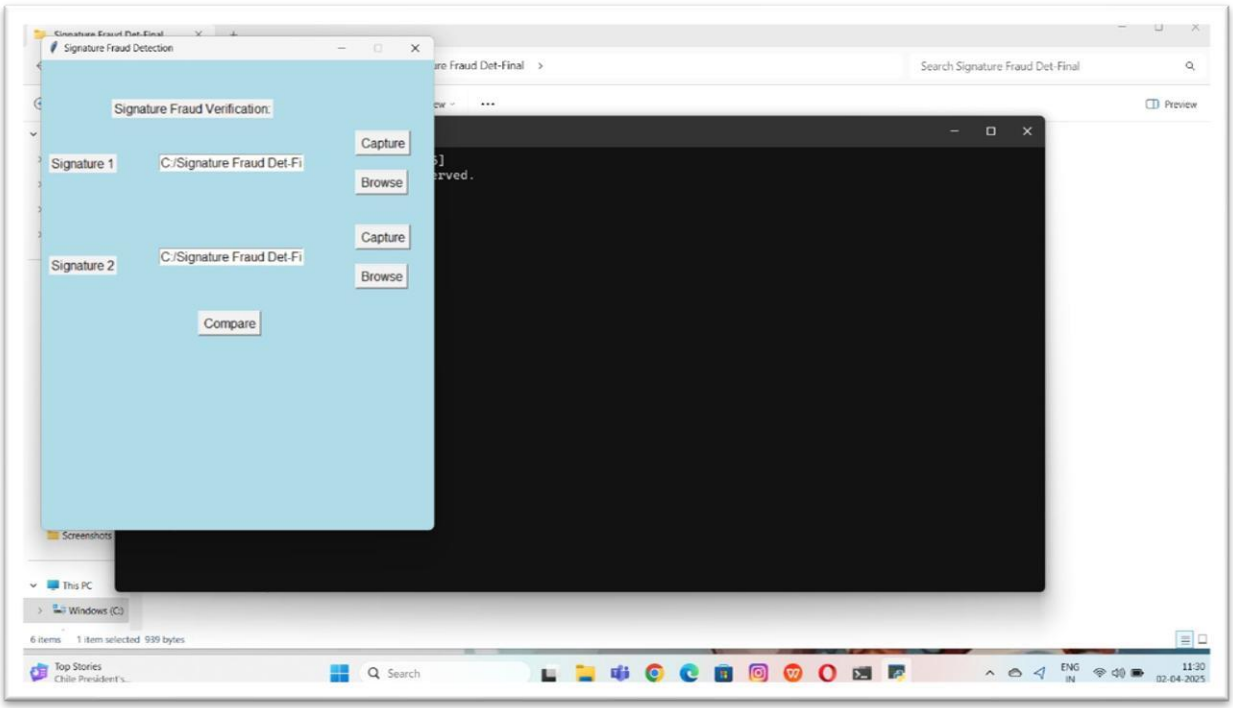

Fig 6.8 Application Process

In this screenshot, the user has selected or browsed two image files for "Signature 1" and "Signature 2." Both fields now show paths pointing to the same directory, indicating that images are loaded. The application is ready for signature comparison using the "Compare" button. This step precedes the fraud detection analysis.

STEP-9

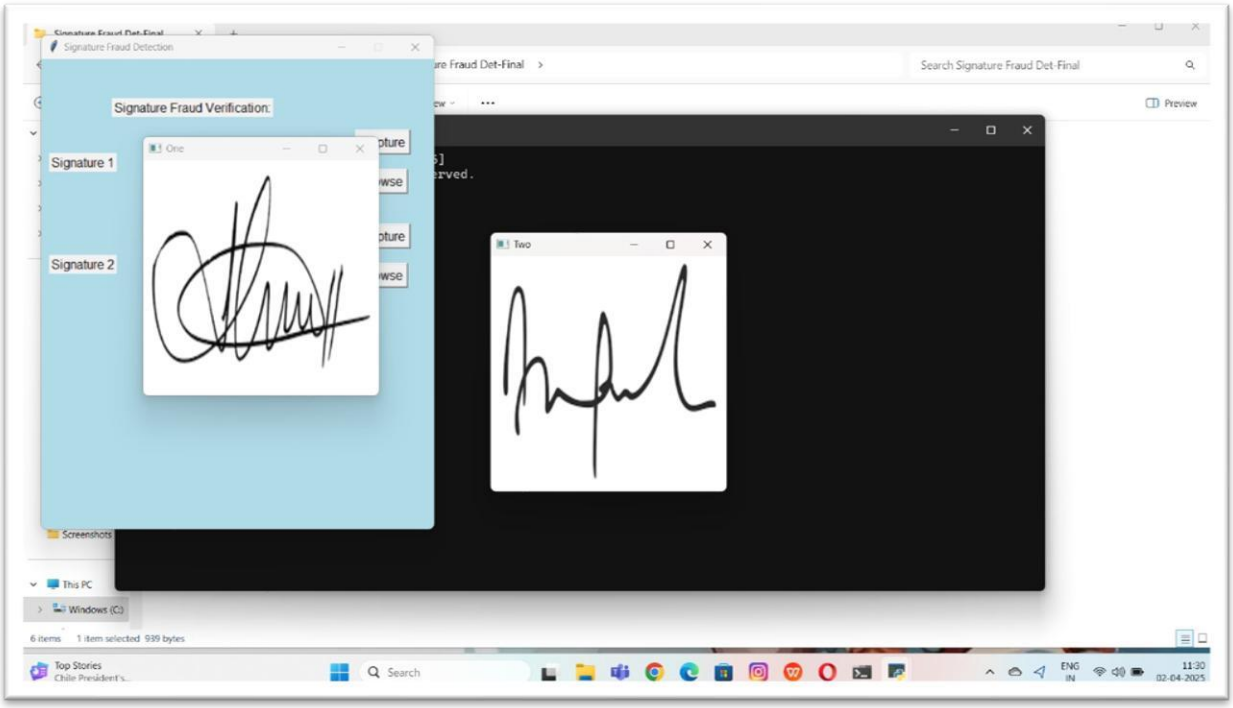

Fig 6.9 Comparing Images

The interface shows a signature verification application. Two signature images labelled “One” and “Two” are displayed side by side, captured or loaded using the "Capture" or "Browse" buttons. The images appear similar at a glance, suggesting they are used for comparison in a fraud detection process.

STEP-10

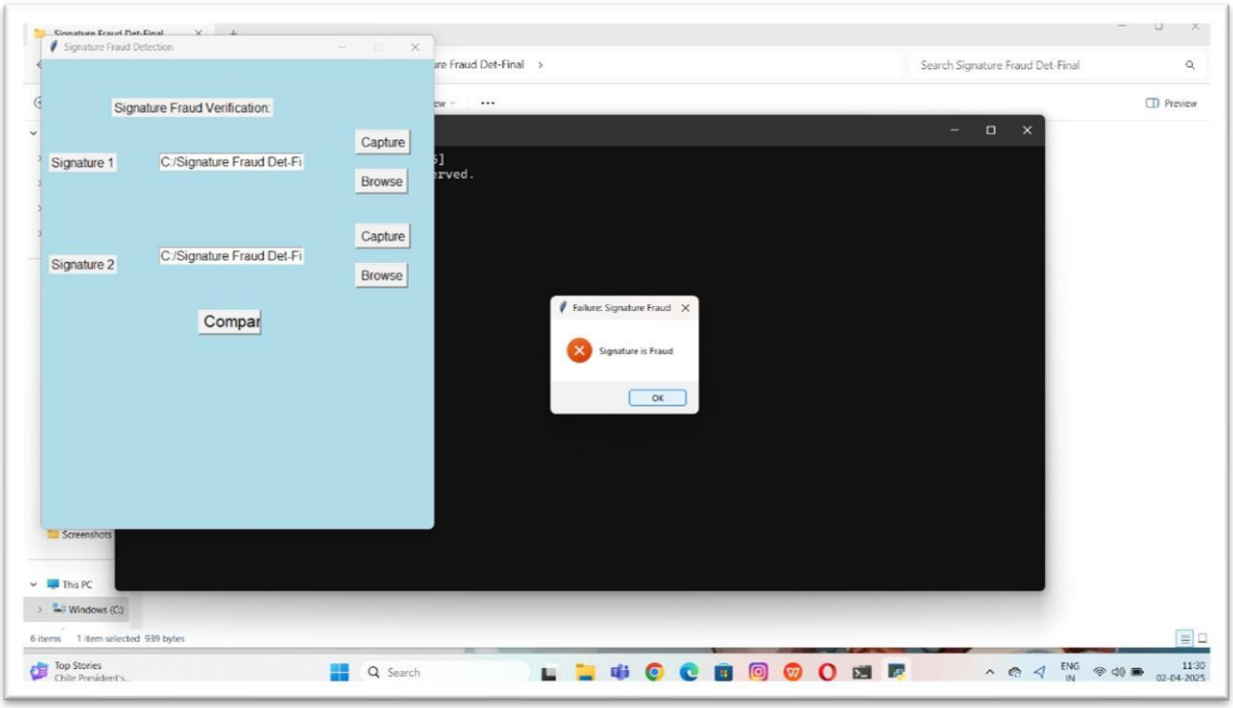

Fig 6.10 Signature Verification

In this image, the same application performs a comparison between the two uploaded signatures. Upon clicking the “Compare” button, a dialog box appears showing the result: “Signatures are 100.0% similar!!” indicating a perfect match. The interface confirms the similarity score and suggests the signatures are likely genuine copies.

STEP-11

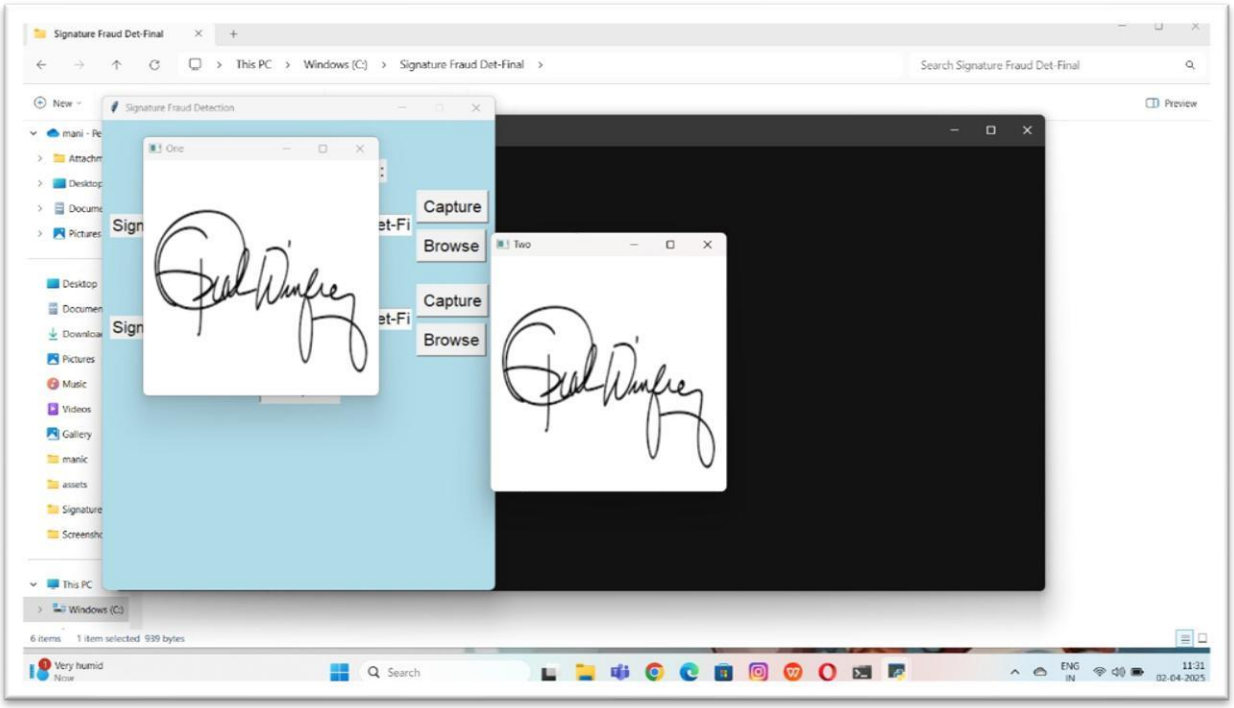

Fig 6.11 Comparing Images

This image shows a graphical user interface (GUI) for a “Signature Fraud Detection” system. Two signature images, labelled “Signature 1” and “Signature 2,” are loaded into the application. Each signature is displayed in a pop-up window for visual verification. Buttons labelled “Capture” and “Browse” allow the user to input or upload signatures for comparison.

STEP-12

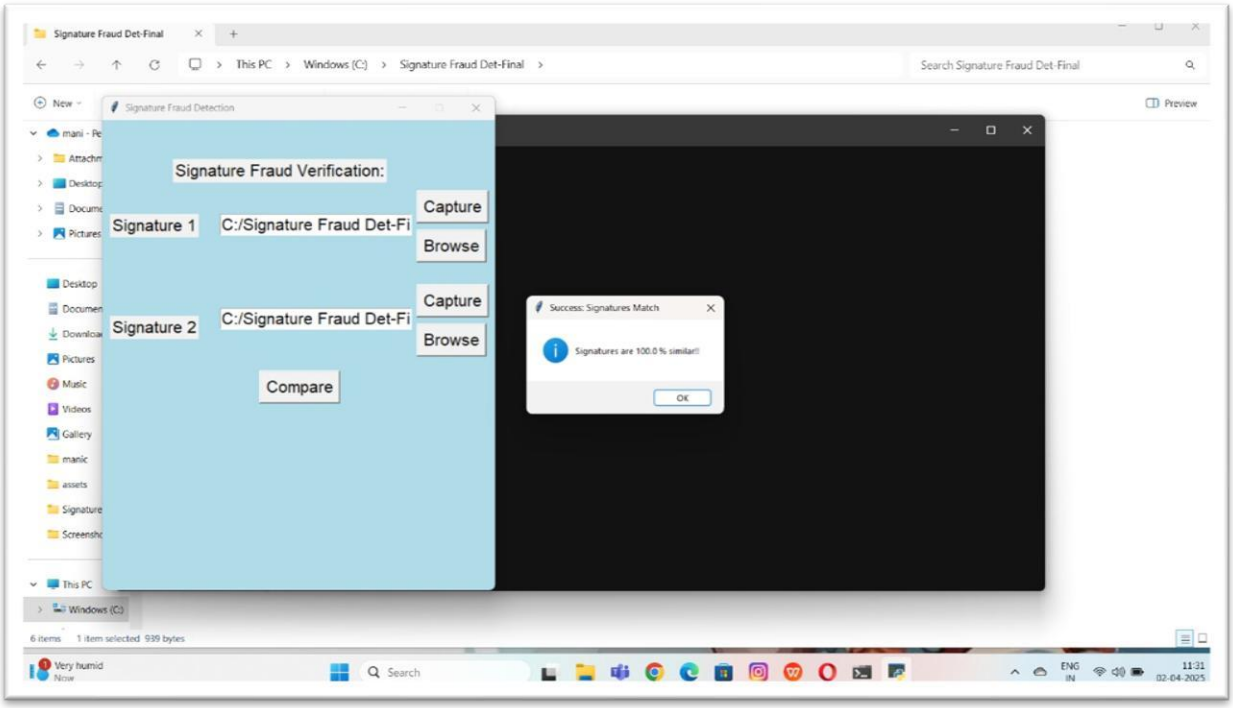

Fig 6.12 Signature Verification

In this image, the same application performs a comparison between the two uploaded signatures. Upon clicking the “Compare” button, a dialog box appears showing the result: “Signatures are 100.0% similar!!” indicating a perfect match. The interface confirms the similarity score and suggests the signatures are likely genuine copies.

ANNEXURE-1

Title of theProject	AUTOMATED DETECTION OF FRAUDLENT SIGNATURE'S USING MACHINE LEARNING
Name of the Student	K.MANI CHAITHANYA(22751A3127) N.NIKITH (22751A3137) V.THIRU KUMARAN(22751A3169) K.BHARATH (22751A3128)
Name of the Guide and Designation	Dr. KARUNIA KRISHNAPRIYA Assistant Professor

TABLE1:OUTCOMEATTAINEDANDITS JUSTIFICATION

PO	JUSTIFICATION
PO1	Appliedtheknowledgeinconceptsofmanagementresearchinchoosingthetopic **"AUTOMATED DETECTION OF FRAUDLENT SIGNATURE'S USING MACHINE LEARNING"**
PO2	Collectedsuitableliterature,primary/secondarydatatoidentifytheproblem basedoncritical thinking
PO3	Employedhonestmethodsindatacollection,analysisandreportpresentationand ensuredunbiasedresults.
PO4	Preparedtheresearchreportinappropriateformatsuggestedandpresentedthe researchworkinaneffectiveway.
PO5	Theresearchobjectiveswereframedinconsistencywiththetileofthe study.
PO6	AppliedtheknowledgeinHRbyunderstandingtheimpactoftrainingand developmentonemployeeperformanceandproductivity
PO7	xxxxxxxxxxxxxxxxxxxxx
PO8	xxxxxxxxxxxxxxxxxxxx